# Introduction

## What is Angular?

Angular is a TypeScript-based front-end framework developed by Google for building SPAs.

## Angular vs AngularJS

AngularJS uses JS and MVC, Angular uses TS and is component-based with RxJS and AOT.

## Advantages

Strong community, modular structure, RxJS, TypeScript support, performance optimizations.

# Angular Basics

## Components

Building blocks with template, class, style, and metadata.

## Modules

Group of components, directives, pipes, services declared with @NgModule.

## Directives

Structural (*ngIf, *ngFor), Attribute ([ngClass], [ngStyle]), Custom directives.

## Data Binding

One-way, two-way ([(ngModel)]), event binding, interpolation.

## Forms

Template-driven forms (ngModel) vs Reactive forms (FormControl, FormGroup).

# Component Lifecycle

## Lifecycle Hooks

ngOnChanges → ngOnInit → ngDoCheck → ngAfterContentInit → ngAfterContentChecked → ngAfterViewInit → ngAfterViewChecked → ngOnDestroy.

## Constructor vs ngOnInit

Constructor for DI, ngOnInit for initialization logic.

## ngOnDestroy

Used to clean resources like unsubscribing observables, stopping intervals.

# Change Detection & Performance

## Change Detection

Angular updates DOM when data changes.

## Strategies

Default (checks entire tree) vs OnPush (checks only when inputs change).

## Pipes

Pure (runs only when input changes) vs Impure (runs every cycle).

## Optimizations

Use trackBy in *ngFor, lazy loading, OnPush, unsubscribing observables.

# Routing & Navigation

## Basics

RouterModule defines routes with path and component.

## ActivatedRoute vs Router

ActivatedRoute gives current route info, Router is used to navigate.

## Route Guards

CanActivate, CanDeactivate, Resolve for navigation control.

## Lazy Loading

Load modules on demand using loadChildren.

# RxJS & Observables

## Observables vs Promises

Promises → single value, Observables → multiple, cancellable.

## Subjects

Subject: emits after subscription, BehaviorSubject: latest value, ReplaySubject: history.

## Operators

map, switchMap, debounceTime, distinctUntilChanged for async handling.

## Use Cases

Search debounce, multiple API calls, event handling.

# HTTP & Interceptors

## HttpClient

Used for GET, POST, PUT, DELETE returning Observables.

## Error Handling

Use catchError to manage errors globally.

## Interceptors

Middleware for auth tokens, logging, API error handling.

## Caching

Use shareReplay to cache results.

# Advanced Angular

## Dynamic Components

Load components dynamically with ViewContainerRef and ComponentFactoryResolver.

## Angular Universal (SSR)

Server-side rendering improves SEO and load speed.

## State Management

Use services, RxJS, or libraries like NgRx/Akita.

## AOT vs JIT

AOT: build-time compilation (faster), JIT: runtime compilation.

## NgZone

Controls Angular change detection for async tasks.

# Testing

## Unit Testing

Use Jasmine for writing tests, Karma as test runner.

## Component Testing

Use TestBed to configure testing module.

## Mocking

Mock services and HTTP requests for isolation.

## E2E Testing

Use Protractor or Cypress for end-to-end tests.

# Deployment & Optimization

## Build

ng build --prod creates optimized build.

## Tree Shaking

Removes unused code from final bundle.

## Differential Loading

Generates modern and legacy bundles.

## Service Workers

Used for PWA offline support and caching.

# Scenario-Based

## Data Sharing

Use shared service with Subject/BehaviorSubject or parent-child @Input/@Output.

## Unsubscribing Observables

Use takeUntil with Subject or async pipe.

## Dirty Forms

Use CanDeactivate guard to prevent navigation.

## Debounced Search

Use debounceTime + switchMap with RxJS.

## Route Param Change

Use ActivatedRoute.paramMap subscription.

# Quick Revision Q&A;

## Most asked Qs

40-50 concise Q&A; for quick revision before interview.